

Prove it, don't just install it.

How a small team builds an unsupervised bot detector with an AI agent crew, and why nothing ships until the claim behind it has survived a measurement.

SUBJECT

Bots Without Labels

DISCIPLINE

Evidence-first development

AUDIENCE

Engineers & teams adopting AI agents

CONTENTS

01 Why a methodology at all

02 Three pillars

03 Evidence over assertion

04 The human-owned crew

05 Aspiration → enforcement

06 The setup, in detail

07 The detector's architecture

08 Branching & delivery pipeline

09 The 13-point checklist

10 Adopting it yourself

01 Why a methodology at all

Most software either works or it doesn't, and a test suite can usually tell the two apart. Our project sits in an awkward corner where that stops being true. Bots Without Labels hunts for automated traffic in server logs that nobody has labelled. No column says which rows came from a botnet. There is no answer key, and there never will be one, because the whole point of the tool is to work on logs where the truth is unknown.

Take a concrete case from our own history. Early on we built a per-entity diversity feature. The reasoning was sound: an automated actor repeats itself, so score every actor in the log by how self-similar its events are, and the monotonous ones are your bots. On a real botnet

capture it lifted recall from almost nothing to 0.998, which felt like a triumph for about an hour. It also flagged more than a fifth of the log, because a nightly backup job hammering one server is every bit as monotonous as a command-and-control beacon. Nothing crashed. Every test stayed green. The feature was still unusable on its own, and the only way to know that was to compute a number no test suite computes.

That is the shape of the problem. When you cannot grade an answer directly, a second and much quieter failure mode opens up alongside the familiar one, and it is the quieter one that will hurt you.

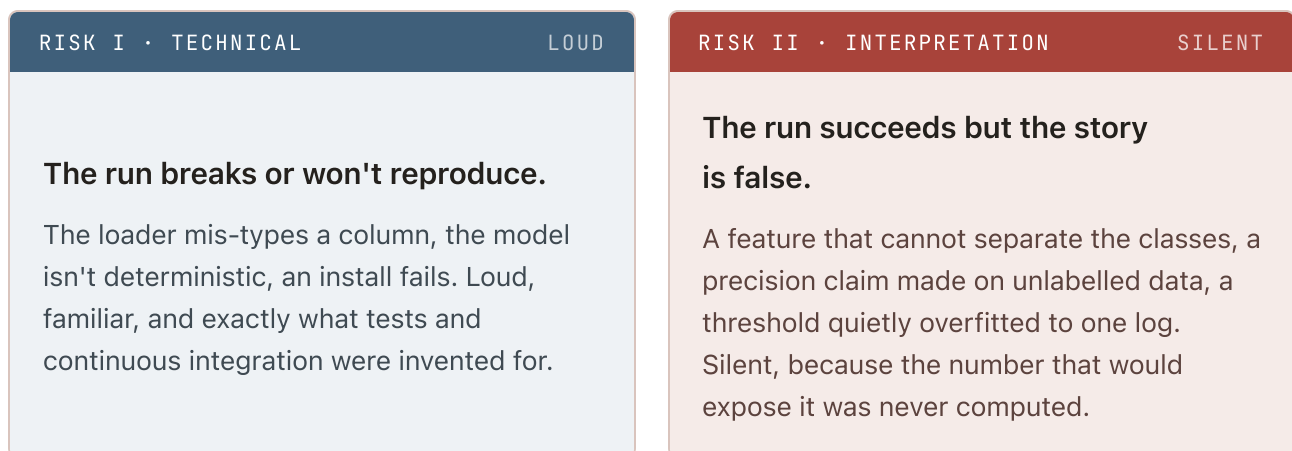


Figure 1. Two kinds of risk. Ordinary engineering discipline covers the left panel. The right panel is what this methodology exists for: it manufactures the evidence that would otherwise never exist, and refuses to let a claim through without it.

The rest of this guide describes that methodology in three parts: how the team decides what is *true*, how it decides what is *done*, and how it makes those rules *bind* rather than merely exist. Every number in these pages comes from the project's own committed records. None were invented for the occasion.

02 Three pillars

The methodology is not a document you read once and file away. It is three interlocking commitments, and each is worthless without the other two.

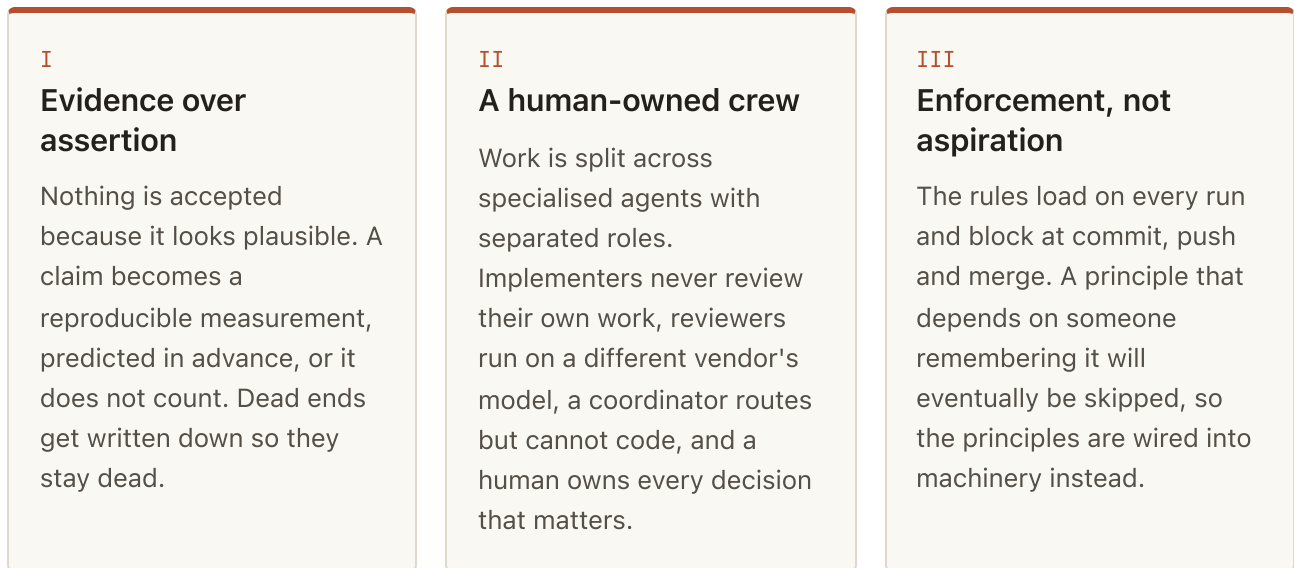


Figure 2. The three pillars. The first is epistemics, the second is organisation, and the third is what keeps the other two honest on a busy Friday afternoon.

03 Pillar I: evidence over assertion

The governing instinct comes from the project's own proof toolkit, and it is worth quoting in full: *do not accept a change because a number moved. Prove why it moved, and predict where it breaks.* Six working practices turn that sentence into daily habit.

Predict before you run

Before any measurement, the expected result goes on the record: a number, an inequality, at minimum a direction. If the measurement matches, you understood the mechanism. If it surprises you, the surprise is the finding, and you investigate it rather than quietly updating your story to fit.

The clearest payoff we have come from a degree-based detection rule. When it first shipped, the rule was direction-agnostic: it flagged any network endpoint whose connectivity was wildly one-sided, without asking whether the node was doing the fanning-out or being fanned into. The design note filed with that commit already named the risk in plain words: such a rule cannot tell a spam bot broadcasting to thousands of victims from a perfectly innocent DNS resolver that thousands of clients happen to query. On the next botnet capture we tested, the prediction arrived on schedule. Precision collapsed to 0.056,

and the rule alone fired on 34,995 rows, of which 33,025 were false alarms. Because the failure had been predicted, nobody had to go hunting for the cause. The fix was structural, not a tuning exercise: narrow the rule to the fan-out side only. Precision recovered to 0.929 with recall held at 0.985, and not one threshold was touched along the way. A predicted failure is cheap. An unpredicted one that you rationalise afterwards is how overfitting hides.

The discipline in one line.

Write down the number you expect before you run. Whatever gap opens between that number and the one you get is the finding.

Prove why, not just that

Headline metrics are properties of the whole system, but the only levers you can pull are individual components, so a moving metric has to be attributed before it can be acted on. The toolkit's main instrument for this is the *counterfactual*: re-score every decision with one rule's evidence surgically removed, and see which flags fall away. A row that stops being flagged was carried by that rule; count how many of those were true positives and how many were false, and you know exactly what the rule is buying and costing you.

This once saved a new rule from being wrongly executed. Midway through a calibration effort, precision on a botnet benchmark sat at a dismal 0.041, and the natural suspect was the freshly added degree rule. The counterfactual said otherwise: the new rule had fired on all 2,000 botnet flows with zero false fires, and it alone carried 1,774 of them. The real culprit was an older monotony rule chewing happily on two degenerate columns (the protocol and TCP-state fields, which hold three or four distinct values each and identify nobody). Fixing the actual cause took precision from 0.041 to 0.978 while recall stood still. Without attribution, we would have gutted the best rule in the system and left the guilty one in place.

A second instrument answers an even earlier question: is this feature worth tuning at all? The *oracle ceiling* sweeps a threshold with the labels visible, which you could never do in production, to find the best precision the feature could possibly reach. That per-entity diversity feature from section 01 had an oracle ceiling of 0.144: no threshold anywhere could do better, because a busy legitimate channel is genuinely as monotonous as a

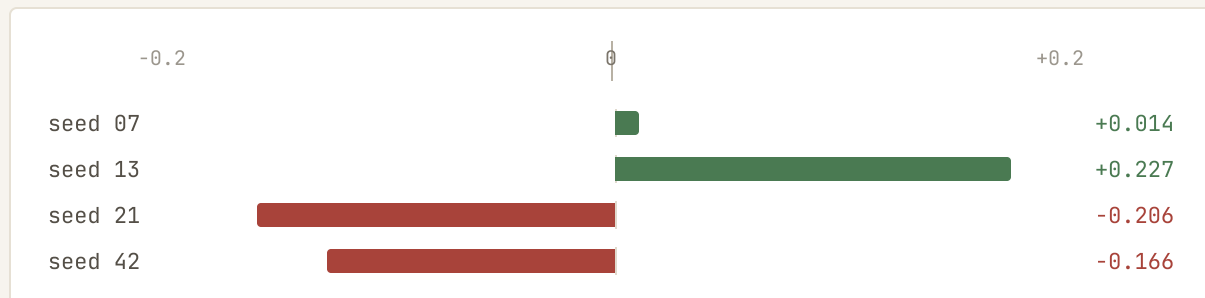
beacon. The low ceiling was not a tuning problem, it was the feature telling us to go find a second, orthogonal signal. We did (relational hub structure), and stacked precision to 0.441, then to 0.846 after timing calibration. The lesson generalises: a low ceiling means change the feature set, not the threshold.

Record the negatives

Ideas that fail are written up with the same care as ideas that ship, because the expensive thing is not the failed experiment. It is the second person who runs it again in six months, from scratch, with fresh optimism. Our findings log holds entries like the periodicity study, which established that on the captures we own, benign traffic is intensely periodic at exactly the cadences bots use (on one capture, 756 benign channels were strongly periodic against a single bot channel at the same 3.7-second beat, which would make a lone periodicity rule 756 false alarms per catch). That paragraph has already stopped the idea being rebuilt twice.

CASE STUDY · A PLAUSIBLE WIN THAT WASN'T

A standing to-do proposed quantile-ranking the numeric features to tame their heavy tails before anomaly scoring. The first measurement, on one data split, looked like a modest win: precision up 0.014. Our rules do not accept one number, so the transform was re-measured across four sampling seeds. The seeds told a different story:



The sign swings by ± 0.2 depending on nothing more than which benign rows the sample happened to draw. The first number was noise wearing a suit. The idea was recorded as a measured wash, the to-do item was closed, and the ledger gained one more entry that will save a future afternoon.

Tier the evidence

Not all proof weighs the same, and the tiers are never allowed to blur. A result on synthetic data proves only that the detector recovers what it planted; the synthetic generator shares the detector's assumptions, so this is the weakest tier, however green it looks. Independent captures labelled by outside researchers, such as the CTU-13 botnet captures (García, Grill, Stiborek & Zunino, 2014) and the CICIDS2017 traffic (Sharafaldin, Habibi Lashkari & Ghorbani, 2018) against which the detector is benchmarked, sit a rung higher. Higher still is a finding that an adversarial reviewer actively tried to refute and could not. Only the top tier may be quoted outside the project, and even then it travels with its licence and its caveats attached.

T4	Citable externally — the top tier, quoted with licence, citation and caveats	PUBLISHABLE
T3	Cross-model refuted — an adversarial reviewer tried to break it and failed	STRONGER STILL
T2	Real, externally labelled — an independent capture the detector never trained on	STRONGER
T1	Synthetic recovery — the detector finds what it planted; it shares its own assumptions	WEAKEST

Figure 3. The evidence ladder. A claim is always labelled with the tier it has actually reached, never the one it aspires to. "Recall 1.0 on synthetic data" and "recall 1.0 on an independent botnet capture" are different sentences, and the methodology forces you to say which one you mean.

Refute across models

The implementer and the reviewer are deliberately different AI vendors. When one model writes a detection change, a different model is handed the diff and told to break the reasoning. Two systems that fail in correlated ways will cheerfully agree on the same wrong answer; two that fail differently catch each other. When they disagree on a blocking issue, the disagreement goes up to the human. It is never averaged away.

A refactor must be bit-identical

The last practice is a sharp line that catches a common self-deception. A change billed as a refactor may not move any output at all, and the proof is a fixed seed and a diff of the full benchmark report, not "the tests still pass". Tests allow ranges; a refactor is not allowed to move the number by one digit in the last decimal place. We ran into this line recently while trying to speed up feature building. A vectorised rewrite of the entropy computation ran three times faster, and its output differed from the original by at most 4.4×10^{-16} , across 1,179 of 7,498 groups. That is a rounding whisper, far below anything a benchmark would notice on a good day. It still fails the contract, because a whisper of drift can flip a threshold tie somewhere, someday, and then a "safe refactor" has silently changed behaviour. The rewrite was reclassified as a behaviour change, parked with its re-entry conditions written down, and the codebase kept its slower, exactly-reproducible loop. The label you put on a change decides which gates it must pass, so the label has to be honest.

04 Pillar II: the human-owned crew

The work is not one giant prompt fired at one giant model. It is decomposed across a small crew of specialised agents, held together by a rule that does most of the heavy lifting: whoever implements does not review, and whoever coordinates does not implement.

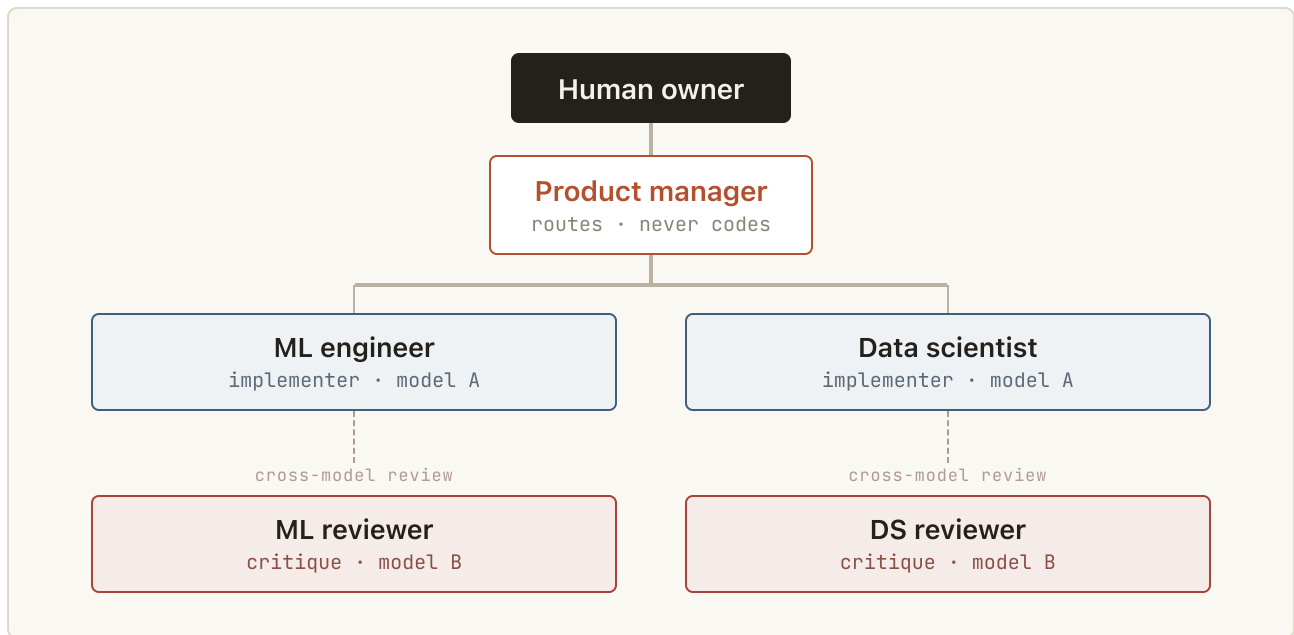


Figure 4. The crew. A human owns direction. A coordinator routes tasks and owns the commit, but writes no code. Two implementer-reviewer pairs cover the engine and the interpretation, and each reviewer runs on a different vendor's model from the implementer it checks (the dashed links). The coordinator boundary is the load-bearing control: the moment the coordinator starts editing files, it becomes an unreviewed implementer and the whole review model quietly collapses.

Why two pairs, not one

The project has two distinct kinds of failure surface, so it has two pairs. The engineering pair owns the detection engine: loader, features, rules, scoring, pipeline, tests. The data-science pair owns the interpretation: the literature grounding, the analysis notebook, and the honesty of every sentence written about what the numbers mean. The split matters because a change can be sound in one domain and rotten in the other. Raise a detection threshold and you have changed the code, the flagged population, and the story the analysis tells a reader, all at once. Such a change crosses both pairs before it lands, and each pair asks its own questions.

The gate every change passes through

Nothing lands on the strength of a summary. Reviewers inspect the actual repository diff and the actual generated artefacts, because an implementer's description of a change is a claim, and claims are what this whole apparatus exists to check. Two checklists bracket every task. A *Definition of Ready* must be satisfied before work begins: the objective, the affected files, the acceptance criteria, the exact validation commands, and what is

explicitly out of scope. A *Definition of Done* must be satisfied before it closes: evidence attached, reviewer approval explicit, no unrelated files staged, residual risks named in writing. If a task cannot fill in the Ready checklist, the correct move is to ask, not to guess.

Skills carry the knowledge; memory stays humble

Deep, hard-won knowledge lives in a library of trigger-loaded *skills*: one holds the architecture contract, one the proof toolkit, one is a chronicle of every rejected idea. Each surfaces exactly when a task touches its subject, so context arrives assembled for the moment rather than dumped wholesale. A separate file-based memory carries working notes between sessions, but it is deliberately second-class: before acting on any remembered fact, an agent verifies it against the current repository, because the repository can change while a memory cannot.

Rule 6 · repository truth beats memory.

Memory and chat transcripts are useful working context. They are not durable truth. A decision that matters is reflected in committed code, tests, docs, or the findings log, or it did not really happen.

05 Pillar III: from aspiration to enforcement

A team can write excellent rules and still bypass them, because a rule that lives only in a document depends on a human remembering it at the precise moment they are tired, rushed, or three context-switches deep. The third pillar closes the gap between a rule being defined and a rule being enforced.

Start with what "good" looks like when the machinery is in place. An idea is not allowed to become a commit without passing through prediction, measurement and review, and a dead end exits the pipeline into the findings log rather than into somebody's fading recollection.

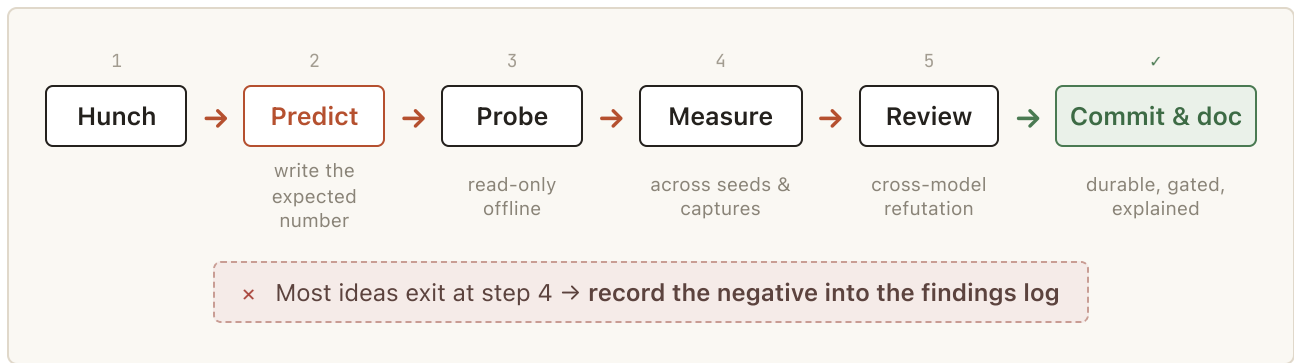


Figure 5. The idea lifecycle. Most ideas exit at step 4 into the findings log, and that exit is a first-class outcome, not a failure. Only what survives measurement reaches review, and only what survives review reaches a gated commit.

Enforcement itself is layered by when it fires, so that each layer catches a different class of mistake at the cheapest moment it can be caught.

EVERY RUN	The always-loaded contract A root entry file distills the non-negotiables so they load for every agent, human and machine, on every session, rather than only when a skill happens to trigger. It is also read natively by the second AI vendor, which previously had no entry point at all.
EVERY COMMIT	Local pre-commit hooks Formatting is checked and a guard refuses direct commits to the main branch, mechanically enforcing a policy that used to rely on everyone remembering it.
EVERY PUSH	Pre-push and continuous integration The full hermetic test suite runs before a push is allowed, and again in the cloud on every push and pull request, on two language versions, with the real machine-learning backend that the quality gates are calibrated against.

Figure 6. The enforcement stack. The same rule (say, the detector must hold its recall floor) is defended at three depths: written in the contract, checked locally before the change can be shared, and re-checked in the cloud where nobody can quietly skip it.

Enforcement earning its keep, on day one.

None of this is theoretical. The very first time the new cloud pipeline ran, it went red. Two detection-quality gates failed with a recall of 0.80 against a floor of 0.85, because the pipeline had installed the wrong machine-learning backend and the system had silently fallen back to a weaker scorer that recovers one fewer bot. Everything had passed on the author's machine, where the real backend happened to be installed already. No document would have caught that. A pipeline that actually runs caught it on the first attempt, and section 8.3 walks through the mechanics of what went wrong.

06 The setup, in detail

The three pillars are the intent. What follows is the machinery that makes them real: the files, the skills, the prompts and the deliberately small toolchain, together with the reason each piece is built the way it is.

6.1 • The repository is the operating system

The first design decision is that the repository, not any agent's memory, is the source of truth. That only works if the repository is legible, so its Markdown files are not incidental notes. They are the operating system the agents run on, and each file has exactly one job.

ENTRY CONTRACTS • load every run

AGENTS.md	The distilled, always-loaded operating contract: eight non-negotiables, the Ready and Done checklists, the build commands. Read natively by the OpenAI (Codex) agents, which have no other entry point.
CLAUDE.md	A thin pointer to AGENTS.md, so the two can never drift apart, plus Claude-specific notes: load the matching skill before acting, and treat memory as context rather than truth.

THE METHODOLOGY CANON • development approach/

shared_agent_principles.md	The full fifteen principles every agent obeys regardless of role: evidence over assertion, role separation, data-science integrity, protection of the working tree.
agentic_development_architecture.md	The rationale: role boundaries, why the coordinator cannot code, why two vendor pairs, where the review gates sit, how the message bus runs.
team_instructions.md • README.md	Canonical operating instructions and the narrative entry point to the whole approach.
community_cheat_sheet.md	The shareable short version: team shape, launch commands, routing. Written so the pattern can travel to other projects.
prompts/ • 5 role charters	One prompt per agent role (section 6.3). The charter each agent is booted with.

THE EVIDENCE LEDGER

evaluation/FINDINGS.md	The chronicle of everything tried, shipped and rejected, with the measured numbers attached. The single most-consulted file before re-attempting any idea, and the compounding asset of the whole method.
evaluation/BENCHMARKS.md	The results registry: recorded metrics per capture, each tagged with its evidence tier and licence. What "the numbers" officially are.

PLAN, ONBOARDING & CROSS-SESSION MEMORY

TODO.md	The roadmap, prioritised in three bands plus lettered follow-ups. Every parked item carries an explicit re-entry trigger, so future work resumes with its context intact instead of being re-derived.
README.md • INSTALL.md • docs/EXEC_MEMO.md	Human onboarding and the executive summary.
memory/MEMORY.md + 11 notes	A one-line index over eleven single-fact memory files: team workflow, launch mechanics, dataset assessments, root causes,

lessons learnt. Working context across sessions, never the source of truth.

Figure 7. The repository as operating system. The layering is deliberate: `AGENTS.md` distils `development approach/`, the skills go deeper still, and `FINDINGS.md` is the durable memory. A newcomer, human or agent, can reconstruct the entire state of the project from these files alone.

6.2 · The skills library: knowledge that loads itself

A single giant instruction file forces an impossible trade. Keep it short and it is too shallow to help; make it complete and it burns the entire context budget on every trivial task. The project resolves this with sixteen *skills*. Each is a folder holding a `SKILL.md` whose description is written as a list of trigger phrases, the questions and symptoms that should summon it: "why is precision below the base rate?", "can I commit this?", "have we tried this before?". The harness matches tasks against those descriptions and loads the right skill at the right moment, so deep knowledge arrives assembled for the occasion instead of dumped in bulk. A few skills also ship executable `scripts/`, such as a detection tracer and a citation-consistency checker, so the knowledge comes with its own instruments.



Figure 8. The sixteen skills, grouped by function. This is Pillar I made operational: the proof toolkit, the evidence bar and the failure archive are not aspirations in a handbook. They are loadable playbooks that appear precisely when a task would otherwise cut a corner.

6.3 · Role prompts over a message bus

The crew in Figure 4 is instantiated concretely. Each role boots from a prompt file and runs as a process on a lightweight local message bus called HCOM. A role prompt is a proper charter, not a one-line persona. The ML engineer's prompt, to take one, fixes what the role owns (loader, features, rules, scoring, pipeline, tests), what it must be able to do (entropy and velocity features, log transforms for skewed distributions, unsupervised anomaly models and their tuning), and the engineering standard it is held to: Google-style Python, no bare `except`, no mutable default arguments, type hints throughout, hermetic tests, executable logic behind `main(argv)`. The same prompt names its reviewer and states that the reviewer runs on a different vendor's model, so the cross-model check is baked into the role rather than bolted on.

Agents launch and route by tag. HCOM earns its place for one reason: it lets heterogeneous CLIs, Claude and Codex in our case, coordinate over a shared SQLite bus without a line of bespoke integration between them.

HOW THE CREW IS ACTUALLY LAUNCHED AND ROUTED

```
# boot each role from its charter prompt
hcom open --tag ml-engineer-          --prompt ".../ml_engineer_prompt.md"
hcom open --tag ml-engineer-reviewer- --prompt ".../ml_engineer_reviewer_prompt.md"# route
Goal: <one sentence>  Scope: rules/scoring/tests
```

The human owner sits on the same bus as a peer, but listens event-driven rather than polling, and the coordinator pushes status upward unprompted. Every durable outcome still has to land in the repository. The bus is for coordination; it is never the record.

6.4 · The toolchain, and why each piece

The toolchain is deliberately small. Every tool on it earns its place by removing one class of "trust me" from the process.

TOOL	ITS JOB	WHY THIS ONE
<code>uv</code>	Environment & dependency manager	One lockfile gives byte-reproducible installs; optional extras keep the base install light; <code>uv run</code> executes hermetically, so "works on my machine" stops being a defence.
<code>isortree</code> (EIF)	The anomaly-scoring backend	Installed as the optional <code>EIF</code> extra so a first install stays small. When absent, the pipeline degrades to a transparent fallback scorer. That seam is exactly where the day-one CI failure lived, which is why CI now pins the real backend.
<code>pytest</code>	The test gate	Hermetic unit and synthetic tests run everywhere. The real-data benchmarks skip themselves when their gitignored captures are absent, so a single suite is green in CI and thorough on a machine that holds the data.
<code>black</code> · <code>pylint</code>	Formatting & linting	Deterministic and argument-free, so no human ever spends judgement on formatting again.
<code>pre-commit</code>	Local enforcement	Stage-scoped hooks: formatting plus a no-commits-to-main guard at commit time, the full suite at push time.
GitHub Actions	Continuous integration	Runs the gates in the cloud on every push and pull request, on two Python versions, with the real EIF backend, where nobody can quietly skip them.
<code>HCOM</code>	Agent message bus	A small local SQLite bus that lets Claude and Codex CLIs coordinate by tag with no custom bridge.
<code>git</code>	Change history	A branch per change, fast-forward merges to main, no direct commits to main: a linear, auditable trail.

Read together, the four parts interlock. The repository holds the truth. The skills deliver the right depth of it on demand. The prompts and the bus put specialised agents to work against it with separated roles. And the toolchain keeps the whole arrangement

reproducible and self-checking. None of the pillars in sections 02 to 05 would survive contact with a busy week without this machinery underneath them.

07 The detector's architecture

Everything so far describes how the team works. This section describes what the team builds, and the two turn out to be the same shape, because the software is architected around the same commitments: explainable, deterministic, and honest about what it does and does not know. A methodology that preached evidence while shipping a black box would be lying to itself.

7.1 · The data-flow pipeline

A log enters as raw bytes and leaves as a per-row decision with a reason attached. In between, it flows through ten small modules in one direction, and, at the centre of the design, it is scored twice in parallel by two independent detectors whose verdicts are then combined.

The first stage sets the tone for everything downstream. The loader does not ask you to describe your log. It sniffs the format, then assigns every column a *role* by inspecting its values. A column of recurring, digit-and-separator tokens such as `10.0.4.117` reads as an actor identifier, someone who can be baselined and graphed. A column that only ever says `tcp`, `udp` or `icmp` reads as a bounded vocabulary; it describes events but identifies nobody, and letting a column like that into the actor analysis is precisely the mistake that once dragged a benchmark's precision down to 0.041. Nothing downstream refers to a column by name, only by role, which is what gives the detector a fighting chance on a log shape it has never seen.

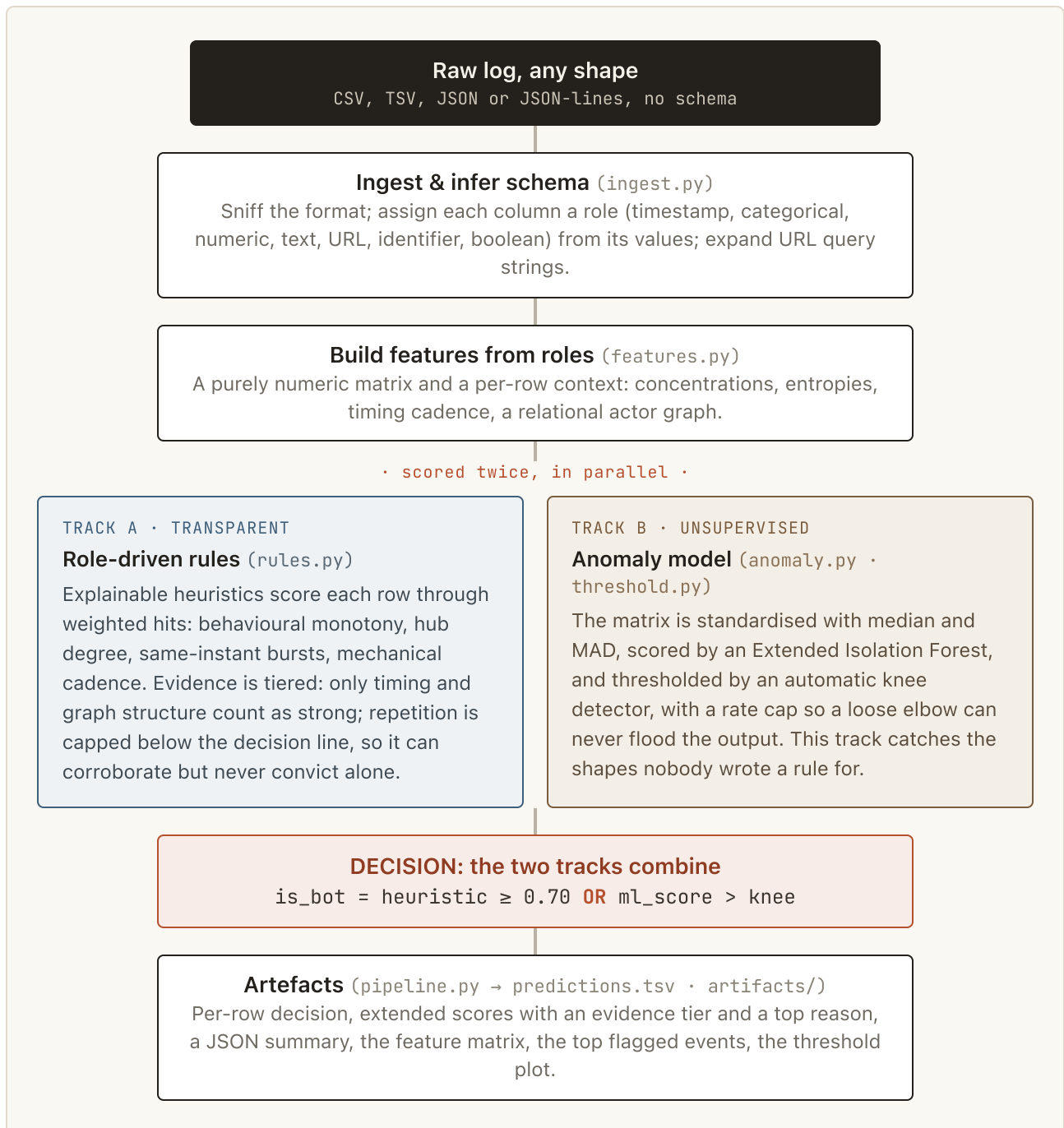


Figure 9. The detector's data flow. Ten modules, one direction. The parallel split into Track A and Track B, and their recombination at the decision node, is the load-bearing idea; the text below explains why.

7.2 · Why two tracks, joined by OR

A transparent rule engine and an unsupervised anomaly model have opposite strengths, and the architecture refuses to pick a favourite. The rules are precise and self-explaining on the shapes we understand well: a beacon's mechanical cadence, a botnet hub's fan-

out. Every flag they raise carries a human-readable reason, which is what makes review possible at all. Their weakness is obvious: they are blind to anything nobody thought to write a rule for. The anomaly model is their mirror image. It needs no prior description of "bot", so it catches the unfamiliar tail, but its verdicts are opaque and it will cheerfully isolate a perfectly benign oddity. Joining the two with a logical OR, so that either detector may flag a row, keeps the recall of the model and the explained precision of the rules, and lets each cover the other's blind side.

Within Track A, the weighting scheme encodes a small piece of hard-won scepticism. Strong signals (timing and graph structure) carry weight 0.70 and sit exactly on the decision line. Everything else is supporting evidence, capped in total at 0.24, so a medium-weight rule at 0.40 plus every scrap of corroboration in the book reaches 0.64 and still falls short of the 0.70 cutoff. Weak evidence can agree with strong evidence; it can never gang up and convict on its own. That arithmetic exists because repetition, the most tempting signal in the data, is also the one benign traffic produces in bulk.

The anomaly side is a stack of established methods chosen on their merits rather than their novelty. Scores come from the isolation-forest family, which rests on a simple and elegant observation: anomalous points are easier to isolate with random splits than normal ones, so the depth at which a point becomes isolated is itself an anomaly score (Liu, Ting & Zhou, 2008). We use the extended variant, which cuts along random slopes rather than axis-parallel planes and thereby removes a directional bias of the original (Hariri, Carrasco Kind & Brunner, 2021). Before scoring, features are standardised with the median and the median absolute deviation rather than the mean and standard deviation, because log features are heavy-tailed and a handful of extreme values would otherwise define the scale for everyone; the 1.4826 factor is the standard normal-consistency constant ($1/\Phi^{-1}(0.75)$) that rescales the MAD to estimate the standard deviation under normality; the MAD estimator itself is due to Hampel (1974), and Rousseeuw and Croux (1993) survey it alongside more robust alternatives. The score threshold is found automatically by the Kneedle knee-detection algorithm (Satopää, Albrecht, Irwin & Raghavan, 2011) and then rate-capped, so the model side can never flag more than a small fraction of rows without rule support. The behavioural diversity features are normalised Shannon entropy (Shannon, 1948). And the relational signals the rules read, a node whose connectivity is anomalously high and one-sided, are the classic near-star outliers of the graph-anomaly literature (Akoglu, McGlohon & Faloutsos, 2010; surveyed in Akoglu, Tong & Koutra, 2015), applied

under the premise, established by BotMiner, that botnets betray themselves through communication structure regardless of protocol or payload (Gu, Perdisci, Zhang & Lee, 2008).

Why this architecture is the methodology.

Four properties of the software are not accidents; each answers to a pillar. It is **deterministic** (fixed seed, single-threaded scoring), so any result reproduces exactly, which is what makes the bit-identical refactor contract checkable. It is **explainable** (every flag carries a reason and an evidence tier), so a reviewer can retrace a decision. It is **schema-driven** (roles, never column names), so a win on one log has a fighting chance of generalising. And its thresholds are **batch-relative** (percentiles of the log in hand), so nothing is secretly tuned to a single capture.

08 Branching and the delivery pipeline

A change is only as trustworthy as the road it travelled to reach `main`. That road is a pipeline of gates, each catching a different class of mistake at the cheapest moment available, and it is arranged so that the same failure would be caught more than once.

8.1 · Branching strategy

The model is short-lived branches over a protected trunk, which is the disciplined core of trunk-based development and of what Humble and Farley (2010) codified as continuous delivery; the evidence that this style of small-batch, gated flow actually predicts delivery performance is laid out in Forsgren, Humble and Kim (2018). In practice it comes down to four habits:

- **Never commit to `main` directly.** Every change starts life on a topic branch (`agent/<topic>`), and a pre-commit hook refuses the alternative, so the policy holds even on the days nobody remembers it.
- **One coherent purpose per commit**, and a negative result is a first-class purpose. Our history contains commits titled "Record quantile-scaling wash" sitting beside feature work, and we consider that a feature of the history, not a blemish.

- **Fast-forward into `main`** after review. History stays linear, and when a benchmark number moves you can walk backwards commit by commit and find the exact step that moved it, with no merge tangles to unpick.
- **The coordinator owns the merge**, and only after review approval or an explicit human waiver. The implementer never lands their own work.

8.2 · The CI/CD pipeline, stage by stage

The same gates run at three depths, which means a regression has three separate chances to be caught before it reaches anyone else. Read the figure left to right; each stage names when it fires and what it catches.

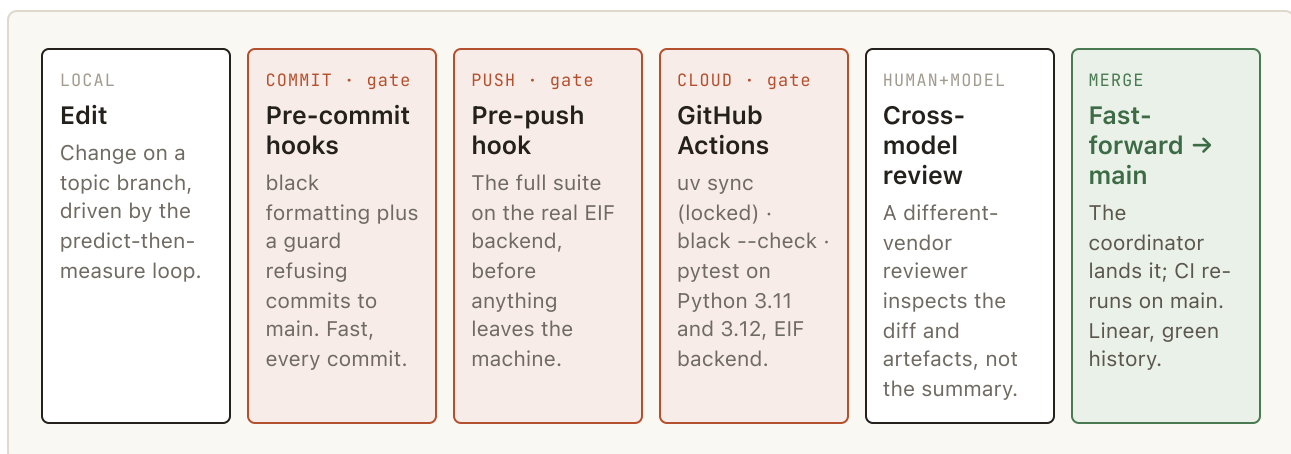


Figure 10. The delivery pipeline. Three of the six stages run the same checks, and that redundancy is the point: the pre-push hook catches a problem before it is shared, and cloud CI catches it again if the local hook was skipped or the local environment differed. This is continuous integration in the literal sense: every change verified on integration, automatically, every time.

8.3 · Two tiers of gate, and the honesty about the second

Not every check can run in the cloud, and the pipeline says so out loud. The hermetic tier, meaning the unit tests, the synthetic-recovery tests and the formatting check, runs everywhere including CI, and is green by construction. The real-data tier, the benchmarks against externally labelled botnet captures, needs datasets that are hundreds of megabytes, licence-bound, and therefore gitignored and never committed. Those tests detect the missing data and skip themselves, so they remain a local, deliberate gate run on

a machine that holds the captures. The contract documents this plainly, because pretending CI covered the real-data tier would manufacture exactly the kind of false green the whole methodology exists to prevent.

The pipeline proved its worth within minutes of existing. Its first run came back red: two detection-quality gates reported a recall of 0.80 against a floor of 0.85. The cause sat in a seam nobody had thought about. The anomaly backend is an optional dependency; when it is missing, the system falls back to a simpler scorer, by design, so that a minimal install still works. CI had synced the project without the optional extra, silently getting the weaker fallback, which recovers one fewer planted bot. On the author's machine everything had passed, because that machine happened to have the real backend installed from an earlier session. The fix took one line, and the lesson took none of the usual arguing: a rule written in prose could never have caught a wrong backend, and a pipeline that actually executes caught it on its first attempt.

Three chances to catch it beats **one polite reminder**.

09 The 13-point checklist

The pillars are the philosophy. This checklist is the instrument: thirteen questions that turn "are we being rigorous?" into a diagnostic you can run against any AI-assisted codebase in an afternoon. Underneath, every question asks the same thing. Is this capability built, or is it improvised each run?

Below is the project's own scorecard, gaps included. Three items were closed during the audit itself, and they are marked with `→ closed`. The one honest weak spot that remains is called out too, because a grade you refuse to write down is a grade you cannot improve.

#	CAPABILITY	GRADE
01	Entry contract — defined, and now loads every run	STRONG
01→	...for both AI vendors — root contract file added	CLOSED
02	Specification first — Ready/Done plus predict-before-run	STRONG
03	Decomposition — skills and crew; hooks added	STRONG
04	Plan & task breakdown — roadmap with re-entry triggers	STRONG
05	Exit criteria & gates — defined, and now enforced	STRONG
06	Context management — assembled on trigger, not dumped	STRONG
07	Sandboxing — convention, not mechanism	PARTIAL
08	Trajectory review — git, findings log, attribution	STRONG
09	Guardrails — stated, and now mechanical too	STRONG
10	Verification loops — predict/measure, cross-model	STRONG
11	Retrieval stack — grounded, cited literature	STRONG
12	Agentic CI/CD — was the real gap; now green	CLOSED
13	Feedback & iteration — negatives recorded and reused	STRONG

Figure 11. The scorecard. What mattered was not the individual grades but their shape. The project was strong on evidence discipline and role separation, and weak in exactly one place: enforcement lagging definition. Excellent rules existed, and a tired human could still bypass every one of them silently. Items 01, 09 and 12 were the fixes, and together they cost half a day.

A rule a tired human can silently skip is **a rule that isn't there.**

The full checklist follows, phrased so you can apply it to your own project. The test is the same for every row: could you point at the machinery, or would you have to trust that everyone remembers?

#	THE QUESTION	WHAT "BUILT, NOT IMPROVISED" LOOKS LIKE
01	Is agent behaviour defined, or improvised each run?	A root entry file loads the non-negotiables every session, for every model, not only when a skill happens to trigger.
02	Is the spec written before the code, or discovered after?	A Definition of Ready precedes work; the expected result is predicted before it is measured.
03	Is the work decomposed, or one giant prompt?	Specialised skills, separated implementer and reviewer roles, and hooks, each doing one job.
04	Can it plan, or only react?	A roadmap with prioritised, sliced tasks and explicit re-entry triggers for parked work.
05	Does it know what "done" means?	A Definition of Done and pinned quality gates, and those gates actually run.
06	Is context assembled, or dumped?	Knowledge loads on trigger for the task at hand; memory is context, never the source of truth.
07	Can a failure be contained, or does it spread?	Isolation (worktrees, read-only probes) so a bad change cannot corrupt shared state.
08	Can you see how it reached an answer?	Linear git history, a findings chronicle, and per-rule attribution tools.
09	Are the boundaries enforced, or assumed?	Hooks and CI that block, not prose that asks politely.
10	Does it check its own work?	Predict-versus-measure, benchmark suites, and adversarial cross-model review.
11	Is knowledge grounded, or hallucinated?	Claims sourced to committed records and cited literature, verified against current code.

#	THE QUESTION	WHAT "BUILT, NOT IMPROVISED" LOOKS LIKE
12	Does it ship through a pipeline, or by hand?	CI runs the gates on every push, on the real backend, where nobody can skip them.
13	Does it improve, or just repeat?	Negative results recorded, so the next person (or the next you) never pays for the same experiment twice.

One caveat stays on the board deliberately. Item 07 remains partial: several agents can share one working tree, protected today by convention rather than by hard isolation. It is the lowest-priority gap, since it only bites under heavy concurrent editing, but the methodology's own rule applies to the methodology too. You name the weak spot; you do not paper over it.

10 Adopting it yourself

You do not need this project's domain to use its method. The transferable core is small enough to state plainly, and none of it requires a botnet.

Convert every claim into a reproducible measurement. "It works", "tests pass" and "this is better" are not results until someone else can re-run them and get the same answer. Write the expected number first. If you find yourself explaining a surprising result after the fact, stop: that explanation is a hypothesis, and it has just earned itself a measurement of its own.

Separate who builds from who checks, and vary the checker. Self-review is not review. Two systems that fail differently will catch far more than one system grading its own homework, and this holds for AI models exactly as it holds for people.

Record the dead ends. The findings log is the highest-compounding artefact in the whole method. Every negative result written down is an experiment the future never has to pay for twice. Ours has already killed the same bad idea more than once, at the cost of a paragraph.

Make the rules load and block, not just exist. The audit's central lesson generalises to any codebase: the gap is rarely knowing the right thing to do; it is that the right thing is bypassable. An entry file that loads every run, a hook that refuses the wrong commit, a pipeline that runs the gates on every push. Definition is cheap. Enforcement is what changes behaviour.

Say which tier your evidence is. Especially where there is no answer key, be ruthless about the distance between "it looks right", "it measured right on my data", and "an adversary tried to break it and could not". Attach the tier to the claim, every time you make it.

None of this slows the work down in the way it first appears to. It front-loads the honesty. The quantile transform that swung by ± 0.2 across seeds, and the wrong backend the pipeline caught on its first run, were both cheap to catch because the machinery already existed, and both would have been ruinously expensive to discover later, after decisions had been built on top of them. That is the entire trade, and it is a good one.

Front-load the honesty. It is cheaper there.

References

The detector's methods are established rather than invented here, and these are the primary sources they stand on. The graph and botnet references were verified against dblp and the publishers' pages before being cited in the project's own findings log; the delivery references ground section 08. As the project's sourcing note insists, these works ground an anomaly signal, not a fraud verdict.

One caveat travels with the evidence, in keeping with the methodology's own rule about naming the tier. The CICIDS2017 capture, one of the two externally labelled botnet captures the detector is benchmarked against, has documented label and feature-extraction issues in its original release (Engelen, Rimmer & Joosen, 2021; Rosay, Cheval, Carlier & Leroux, 2022). Results measured against it are read accordingly, and the CTU-13 captures (CC-BY licensed) carry the weight where the two disagree.

1. Akoglu, L., McGlohon, M., & Faloutsos, C. (2010). OddBall: Spotting anomalies in weighted graphs. In *Advances in Knowledge Discovery and Data Mining (PAKDD 2010)*, Lecture Notes in Computer Science, vol. 6119, pp. 410–421. Springer. doi:10.1007/978-3-642-13672-6_40
2. Akoglu, L., Tong, H., & Koutra, D. (2015). Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery*, 29(3), 626–688. doi:10.1007/s10618-014-0365-y
3. Engelen, G., Rimmer, V., & Joosen, W. (2021). Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In *2021 IEEE Security and Privacy Workshops (SPW)*, pp. 7–12. IEEE. doi:10.1109/SPW53761.2021.00009
4. Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
5. García, S., Grill, M., Stiborek, J., & Zunino, A. (2014). An empirical comparison of botnet detection methods. *Computers & Security*, 45, 100–123. doi:10.1016/j.cose.2014.05.011
6. Gu, G., Perdisci, R., Zhang, J., & Lee, W. (2008). BotMiner: Clustering analysis of network traffic for protocol- and structure-independent botnet detection. In *Proceedings of the 17th USENIX Security Symposium*, pp. 139–154. USENIX Association.
7. Hampel, F. R. (1974). The influence curve and its role in robust estimation. *Journal of the American Statistical Association*, 69(346), 383–393. doi:10.1080/01621459.1974.10482962
8. Hariri, S., Carrasco Kind, M., & Brunner, R. J. (2021). Extended isolation forest. *IEEE Transactions on Knowledge and Data Engineering*, 33(4), 1479–1489. doi:10.1109/TKDE.2019.2947676
9. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
10. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM 2008)*, pp. 413–422. IEEE. doi:10.1109/ICDM.2008.17
11. Rosay, A., Cheval, E., Carlier, F., & Leroux, P. (2022). Network intrusion detection: A comprehensive analysis of CIC-IDS2017. In *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP 2022)*, pp. 25–36.

SciTePress. doi:10.5220/00107740000003120

2. Rousseeuw, P. J., & Croux, C. (1993). Alternatives to the median absolute deviation. *Journal of the American Statistical Association*, 88(424), 1273–1283. doi:10.1080/01621459.1993.10476408
3. Satopää, V., Albrecht, J., Irwin, D., & Raghavan, B. (2011). Finding a "Kneedle" in a haystack: Detecting knee points in system behavior. In *Proceedings of the 31st International Conference on Distributed Computing Systems Workshops*, pp. 166–171. IEEE. doi:10.1109/ICDCSW.2011.20
4. Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423. doi:10.1002/j.1538-7305.1948.tb01338.x
5. Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, pp. 108–116. SciTePress. doi:10.5220/0006639801080116

COLOPHON

This guide describes the working methodology of **Bots Without Labels**, an unsupervised bot and botnet detector for unlabelled logs. Every figure, from the seed sweep to the scorecard to the day-one pipeline failure, is drawn from the project's own committed records and findings log. None were composed for illustration.

The measured numbers quoted are internal benchmark results against externally labelled captures. They are evidence for review, not verdicts, and each is reported at the evidence tier it actually reached. Nothing here should be read as a claim of proven fraud against any real party.

Prove it, don't just install it.